

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO 3: Implement dynamic web applications by utilizing DOM-based client-side scripting and employing Java Servlets for server-side request processing and response handling. (L3)

Assessment Tool : Scenario-Based Assessment

Weightage: 40%

QUESTIONS:

Total Marks: 30

Question 1

Suppose that a college wants to develop an online student registration system where students submit their details through a web form, and the data is processed using a Java Servlet.

Explain the architecture of a servlet-based application. Design and implement a servlet to handle form data using both GET and POST methods. Describe the servlet life cycle (init, service, destroy) and justify which method is more appropriate for handling sensitive data.

Question 2

Suppose that a website requires users to log in and maintain their session across multiple pages after authentication.

Design a session management system using HttpSession and Cookies in Servlets. Explain how session tracking works in web applications and compare session-based tracking with cookies.

Question 3

Suppose that an e-commerce platform needs to store and retrieve product details from a database using a Java-based web application.

Design a JDBC-based application to connect to a database, insert product data, and retrieve it using SQL queries. Explain the JDBC architecture and describe the steps involved in database connectivity along with proper exception handling.

Question 1: Servlet-Based Student Registration System

(a) Architecture of a Servlet-Based Application

A servlet-based web application follows a **three-tier architecture**:

1. **Client Layer (Browser)**
 - The student opens a web page and fills out the registration form.
 - The browser sends an HTTP request (GET or POST) to the web server.
2. **Web Server/Servlet Container**
 - The request is received by a servlet container such as **Apache Tomcat**.
 - The servlet processes the request, performs validation or business logic, and communicates with the database if required.
3. **Database Layer**
 - Stores student information.
 - The servlet uses JDBC to insert or retrieve student records.

Architecture Diagram

Student (Browser)

|

HTTP Request

|

V

Servlet Container
(Apache Tomcat)

|

Java Servlet

|

JDBC Connection

|

V

Database (MySQL)

|

HTTP Response

|

V

Student (Browser)

(b) Student Registration Servlet

HTML Form

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>Student Registration</title>
```

```
</head>
```

```
<body>
```

```
<form action="StudentServlet" method="post">
```

```
  Name:
```

```
  <input type="text" name="name"><br><br>
```

```
  Email:
```

```
<input type="email" name="email"><br><br>

Course:
<input type="text" name="course"><br><br>

<input type="submit" value="Register">
</form>

</body>
</html>
```

Java Servlet

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class StudentServlet extends HttpServlet {

    public void init() throws ServletException {
        System.out.println("Servlet Initialized");
    }

    protected void doGet(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        String name = request.getParameter("name");

        out.println("<h2>GET Request</h2>");
        out.println("Student Name: " + name);
    }

    protected void doPost(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        String name = request.getParameter("name");
        String email = request.getParameter("email");
        String course = request.getParameter("course");

        out.println("<h2>Registration Successful</h2>");
        out.println("Name: " + name + "<br>");
        out.println("Email: " + email + "<br>");
        out.println("Course: " + course);
    }
}
```

```

    }

    public void destroy() {
        System.out.println("Servlet Destroyed");
    }
}

```

(c) Servlet Life Cycle

The servlet life cycle consists of three main methods:

1. init()

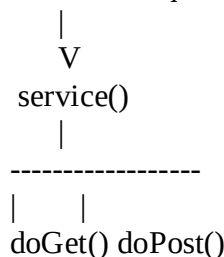
- Called only once when the servlet is first loaded.
- Used for initialization such as database connections or configuration loading.

public void init() throws ServletException

2. service()

- Called for every client request.
- Determines whether to invoke doGet(), doPost(), doPut(), etc.

Browser Request



3. destroy()

- Called once before the servlet is removed.
- Releases resources like database connections and files.

public void destroy()

(d) GET vs POST

Feature	GET	POST
Data Location	URL	Request Body
Security	Less secure	More secure
Data Length	Limited	Large amount
Bookmark	Yes	No
File Upload	No	Yes

Which method is better for sensitive data?

POST is more appropriate because:

- Data is sent in the request body instead of the URL.
 - Passwords and personal information are not exposed in browser history.
 - Supports large amounts of data.
 - More suitable for registration and login forms.
-

Question 2: Session Management Using HttpSession and Cookies

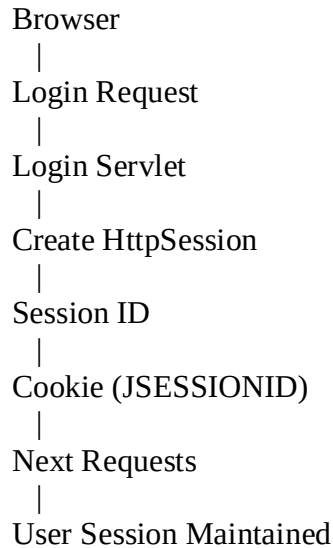
(a) Session Management Design

Login Process

1. User enters username and password.

2. Servlet validates credentials.
3. If valid:
 - Create an HttpSession.
 - Store user information in the session.
 - Optionally create a cookie containing the session ID.
4. User accesses multiple pages using the same session.

Session Diagram



(b) Login Servlet Example

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class LoginServlet extends HttpServlet {

    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response)
        throws ServletException, IOException {

        String username = request.getParameter("username");
        String password = request.getParameter("password");

        if(username.equals("admin") && password.equals("1234")) {

            HttpSession session = request.getSession();

            session.setAttribute("user", username);

            Cookie cookie = new Cookie("username", username);
            cookie.setMaxAge(60 * 60);

            response.addCookie(cookie);

            response.getWriter().println("Login Successful");
```

```

    } else {

        response.getWriter().println("Invalid Login");

    }
}
}

```

(c) Accessing Session

```

HttpSession session = request.getSession(false);

if(session != null){

    String user = (String) session.getAttribute("user");

    out.println("Welcome " + user);

}

```

(d) How Session Tracking Works

HTTP is a **stateless protocol**, meaning each request is independent.

Session tracking maintains user information by:

- Creating a unique session ID.
 - Sending the session ID to the browser (usually in a cookie named JSESSIONID).
 - The browser sends the session ID back with each request.
 - The server retrieves the associated session data.
-

(e) HttpSession vs Cookies

Feature	HttpSession	Cookies
Storage	Server	Client Browser
Security	High	Lower
Size Limit	Large	About 4 KB
Lifetime	Until timeout/logout	Configurable
Sensitive Data	Suitable	Not recommended

Comparison

HttpSession

- Data stored securely on the server.
- Faster access to user objects.
- Automatically expires after inactivity.

Cookies

- Stored in the user's browser.
- Can persist even after browser restart.
- Used to remember user preferences or session identifiers.

Conclusion: Use **HttpSession** for authentication and sensitive information, while cookies are mainly used to store the session ID or non-sensitive preferences.

Question 3: JDBC-Based Product Management System

(a) JDBC Architecture

Java Application

|

JDBC API

|

JDBC Driver Manager

|

JDBC Driver

|

Database (MySQL)

The JDBC API provides a standard interface for Java applications to communicate with different databases through database-specific JDBC drivers.

(b) Steps for Database Connectivity

1. Import JDBC packages.
 2. Load the JDBC driver.
 3. Establish a database connection.
 4. Create a Statement or PreparedStatement.
 5. Execute SQL queries.
 6. Process the ResultSet.
 7. Close all resources.
 8. Handle exceptions properly.
-

(c) Insert Product Data

```
import java.sql.*;
```

```
public class InsertProduct {
```

```
    public static void main(String[] args) {
```

```
        try {
```

```
            Class.forName("com.mysql.cj.jdbc.Driver");
```

```
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/shop",
                "root",
                "password");
```

```
            String sql =
                "INSERT INTO product(id,name,price) VALUES(?,?,?)";
```

```
            PreparedStatement ps = con.prepareStatement(sql);
```

```
            ps.setInt(1,101);
            ps.setString(2,"Laptop");
            ps.setDouble(3,55000);
```

```
            ps.executeUpdate();
```

```
            System.out.println("Product Inserted");
```

```

        ps.close();
        con.close();

    } catch(Exception e) {

        e.printStackTrace();

    }

}

```

(d) Retrieve Product Data

```

import java.sql.*;

public class DisplayProduct {

    public static void main(String args[]) {

        try {

            Class.forName("com.mysql.cj.jdbc.Driver");

            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/shop",
                "root",
                "password");

            Statement st = con.createStatement();

            ResultSet rs = st.executeQuery(
                "SELECT * FROM product");

            while(rs.next()) {

                System.out.println(
                    rs.getInt("id") + " "
                    + rs.getString("name") + " "
                    + rs.getDouble("price"));

            }

            rs.close();
            st.close();
            con.close();

        } catch(Exception e) {

```



```
        e.printStackTrace();
    }
}
}
```

(e) Exception Handling

Proper exception handling ensures that database errors are managed gracefully.

Example:

```
try {
    // Database operations
}
catch (SQLException e) {
    System.out.println("Database Error: " + e.getMessage());
}
catch (ClassNotFoundException e) {
    System.out.println("Driver Not Found");
}
finally {
    // Close ResultSet, Statement, and Connection
}
```

Best practices:

- Use PreparedStatement to prevent SQL injection.
- Catch SQLException and ClassNotFoundException.
- Close ResultSet, Statement, and Connection in a finally block or use **try-with-resources**.
- Display user-friendly error messages while logging technical details.

Code of Conduct:

I Aaron Samuel S(192411022) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand